

Project Submission – Database Administration Vehicle Service Management Database

Student Details

Student Name	PRITHVI . S
Register Number	2428C0388
Department	BSc AI&ML
ID	asbrubw2428c0388
College	AKSHAYA COLLEGE OF ARTS AND SCIENCE
Semester	V
Date of Submission	14-9-2026
TITLE	VEHICLE SERVICE MANAGEMENT DATABASE
GIT HUB LINK	https://github.com/prithviai2005/BRU-.git

1. Problem Statement & Objectives – 5 Marks

Problem Statement

Vehicle service centers handle many customers, vehicles, and repeated service visits every day. When this information is maintained manually or in scattered spreadsheets, it becomes difficult to track which vehicle belongs to which customer, what services were performed, when a vehicle was last serviced, and how much revenue each service type generates. This often leads to lost records, duplicate entries, delayed service history lookups, and poor customer service. The Vehicle Service Management Database is designed to solve this problem by storing customer details, vehicle details, and service records in a structured, relational database. This allows the service centre staff to quickly retrieve a customer's vehicle history, track ongoing and completed services, calculate revenue, and generate reports using simple SQL queries instead of manual bookkeeping.

Objectives

- *To design a normalized relational database for a vehicle service center.*
- *To store and manage customer information, vehicle details, and service records.*
- *To establish proper relationships between tables using primary and foreign keys.*
- *To perform CRUD operations (INSERT, SELECT, UPDATE, DELETE) on the database.*
- *To retrieve meaningful information using filtering, sorting, aggregate functions, and joins.*

- To generate useful reports such as total revenue, service history, and service status summaries.

2. Database Design – 10 Marks

2.1 Database Structure

The database consists of three main tables — Customer, Vehicle, and Service — that together capture the complete workflow of a vehicle service centre: a customer owns one or more vehicles, and each vehicle can have multiple service records over time.

Tables Used

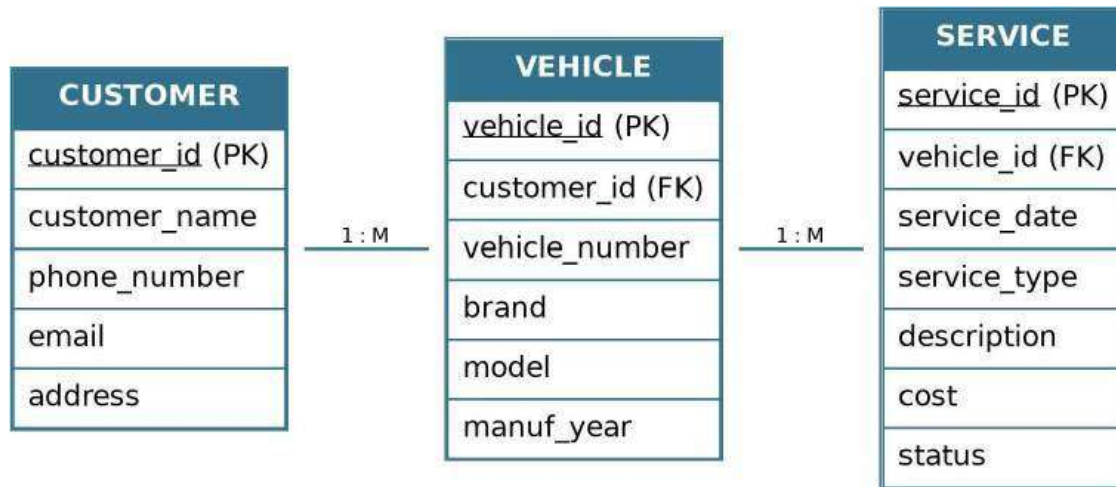
Table name	Purpose
Customer	Stores customer information like(name, phone,email,address)
Vehicle	Stores vehicle information linked to a customer
Service	Store vehicle service records linked to a vehicle

2.2 Entity Relationship Diagram

The ER diagram below shows the three tables, their attributes, primary keys (PK), foreign keys (FK), and the relationships between them (1 Customer → Many Vehicles → Many Services):

Diagram:

ER Diagram



2.3 Technologies Used

Technology / Tool	Purpose
MySQL	Database management system used to store and manage data
MySQL Workbench	Environment used to write and execute SQL scripts
SQL	Used for database creation, manipulation, and querying

3. Practical Implementation – 15 Marks

3.1 Database and table creation

Database Creation

Oracle does not use a CREATE DATABASE statement like MySQL — each user works within their own schema, which is already provisioned in Oracle Live SQL / SQL Developer. Tables are created directly within this schema, as shown below.

```
CREATE TABLE Customer (
customer NUMBER (5) PRIMARY KEY, customer name
VARCHAR2(50) NOT NULL,
phone number VARCHAR2(15) UNIQUE NOT NULL,
email VARCHAR2(50) UNIQUE,
address VARCHAR2(100)

);
```

Vehicle Table

```
CREATE TABLE Vehicle (
vehicle_id NUMBER (5) PRIMARY KEY, customer_id
NUMBER (5) NOT NULL, vehicle number
VARCHAR2(15) UNIQUE NOT NULL, brand
VARCHAR2(30) NOT NULL, model
VARCHAR2(30) NOT NULL,
manuf_year NUMBER (4) CHECK (manuf_year BETWEEN 1990
AND 2026),
CONSTRAINT fk_customer FOREIGN KEY (customer_id)
REFERENCES Customer(customer_id)
ON DELETE CASCADE

);
```

Service Table

```
CREATE TABLE Service (
Service_id NUMBER (5) PRIMARY KEY,
Vehicle_id NUMBER (5) NOT NULL,
Service_date DATE DEFAULT SYSDATE,
Service_type VARCHAR2(30) NOT NULL,
Description VARCHAR2(100),
Cost NUMBER (8,2) CHECK (cost >= 0),
Status VARCHAR2(20) DEFAULT 'Pending'
CHECK (status IN ('Pending', 'In Progress', 'Completed')),
CONSTRAINT fk_vehicle FOREIGN KEY (vehicle_id)
REFERENCES Vehicle(vehicle_id)
ON DELETE CASCADE

);
```

3.2 Constraints and Relationships

Explain the primary keys, foreign keys, and other constraints applied to the tables.

Table	Primary Key	Foreign Key	Other Constraints
Customer	Customer_id	—	Phone number UNIQUE & NOT NULL, email UNIQUE, customer name NOT NULL
Vehicle	Vehicle_id	Customer → Customer (customer id)	Vehicle number UNIQUE & NOT NULL, manu_year CHECK (1990–2026), ON DELETE CASCADE
Service	Service_id	vehicle_id → Vehicle(vehicle_id)	Cost CHECK (>=0), status CHECK (Pending/In Progress/Completed), ON DELETE CASCADE

3.3 Data Insertion

Customer Data

```
INSERT INTO Customer VALUES (101, 'Arun Kumar', '9840012345', 'arun.k@gmail.com', 'Chennai');
INSERT INTO Customer VALUES (102, 'Priya Sundar', '9884023456', 'priya.s@gmail.com', 'Coimbatore');
INSERT INTO Customer VALUES (103, 'Vignesh Raja', '9994034567', 'vignesh.r@gmail.com', 'Madurai');
INSERT INTO Customer VALUES (104, 'Deepa Lakshmi', '9791045678', 'deepa.l@gmail.com', 'Trichy');
INSERT INTO Customer VALUES (105, 'Karthik Subramani', '8608056789', 'karthik.s@gmail.com', 'Salem');
```

Vehicle Data

```
INSERT INTO Vehicle VALUES (201, 101, 'TN01AB1234', 'Honda', 'City', 2020);
INSERT INTO Vehicle VALUES (202, 102, 'TN37CD5678', 'Toyota', 'Innova', 2019);
INSERT INTO Vehicle VALUES (203, 103, 'TN59EF9012', 'Maruti', 'Swift', 2021);
INSERT INTO Vehicle VALUES (204, 104, 'TN45GH3456', 'Hyundai', 'Creta', 2022);
INSERT INTO Vehicle VALUES (205, 105, 'TN30IJ7890', 'Honda', 'Activa', 2018);
INSERT INTO Vehicle VALUES (206, 101, 'TN01KL2345', 'Yamaha', 'FZ', 2023);
```

Service Data

```
INSERT INTO Service VALUES (301, 201, DATE '2026-01-05', 'Oil Change', 'Engine oil and filter change', 1200.00, 'Completed');
INSERT INTO Service VALUES (302, 202, DATE '2026-02-10', 'Brake Service', 'Front brake pad replacement', 2500.00, 'Completed');
INSERT INTO Service VALUES (303, 203, DATE '2026-02-20', 'General Service', 'Full vehicle check-up', 1800.00, 'In Progress');
INSERT INTO Service VALUES (304, 204, DATE '2026-03-01', 'AC Repair', 'AC gas refill and compressor check', 3200.00, 'Completed');
INSERT INTO Service VALUES (305, 205, DATE '2026-03-15', 'Tyre Replacement', 'Rear tyre replacement', 1500.00, 'Completed');
INSERT INTO Service VALUES (306, 206, DATE '2026-04-02', 'Oil Change', 'Engine oil change', 600.00, 'Completed');
INSERT INTO Service VALUES (307, 201, DATE '2026-04-18', 'Battery Check', 'Battery health check and top-up', 450.00, 'Completed');
INSERT INTO Service VALUES (308, 203, DATE '2026-05-05', 'Wheel Alignment', 'Wheel alignment and balancing', 900.00, 'Pending');
COMMIT;
```

3.4 SQL Operations

INSERT

Adds a new customer record into the Customer table.

```
INSERT INTO Customer VALUES (106, 'Meena Rani', '9500067890', 'meena.r@gmail.com', 'Erode');
```

SELECT

Retrieves all records from a table.

```
SELECT * FROM Customer;
SELECT * FROM Vehicle;
SELECT * FROM Vehicle;
```

UPDATE

Updates the status of a service record once it is finished.

```
UPDATE Service
SET status = 'Completed'
WHERE service_id = 308;
```

DELETE

Removes a customer record from the database.

```
DELETE FROM Customer  
WHERE customer_id = 106;
```

WHERE

Filters service records that are marked as Completed.

```
SELECT * FROM Service  
WHERE status = 'Completed';
```

ORDER BY

Sorts service records by cost, highest first.

```
SELECT * FROM Service  
ORDER BY cost DESC;
```

LIKE

Finds customers whose name starts with the letter 'A'.

```
SELECT * FROM Customer  
WHERE customer_name LIKE 'A%';
```

BETWEEN

Retrieves services costing between 1000 and 2500.

```
SELECT * FROM Service  
WHERE cost BETWEEN 1000 AND 2500;
```

Aggregate Functions

Calculates key summary statistics for the Service table.

```
SELECT COUNT (*) AS total services FROM Service;  
SELECT SUM (cost) AS total revenue FROM Service;  
SELECT AVG (cost) AS average cost FROM Service;  
SELECT MAX (cost) AS highest cost FROM Service;  
SELECT MIN (cost) AS lowest cost FROM Service;
```

Groups service records by status, and separately by service type with total revenue per type.

```
SELECT status, COUNT (*) AS total count
FROM Service
GROUP BY status;
SELECT service type, SUM (cost) AS total cost
FROM Service
GROUP BY service type
ORDER BY total cost DESC;
```

INNER JOIN

Retrieves customer, vehicle, and service information together in a single result set.

```
SELECT c.customer_name, v.vehicle_number, v.brand, v.model, s.service_type,
s.service_date, s.cost, s.status
FROM Customer c
INNER JOIN Vehicle v ON c.customer_id = v.customer_id
INNER JOIN Service s ON v.vehicle_id = s.vehicle_id ORDER BY s.service_date;
```

3.5 Project Execution

The database was created in Oracle Live SQL by first defining the Customer, Vehicle, and Service tables with appropriate primary keys, foreign keys, and constraints. Sample data was then inserted into each table using INSERT statements, maintaining referential integrity (each Vehicle references a valid Customer, and each Service references a valid Vehicle). All SQL scripts — table creation, data insertion, and query operations — were written and executed sequentially in the SQL worksheet, and the results of each query (SELECT output, updated rows, join results) were verified directly in the results grid before being captured as screenshots.

4 Output Screenshots – 10 Marks

Include screenshots showing the database creation, table structure, executed queries, and query results.

Screenshot 1 – Database and Tables

The screenshot shows the Oracle FreeSQL web interface. The main editor displays the following SQL code:

```

1  -- VEHICLE SERVICE MANAGEMENT DATABASE
2  -- ORACLE SQL SCRIPT (Run on Oracle Live SQL / SQL Developer)
3  --
4  --
5  --
6  -- NOTE: Oracle doesn't use "CREATE DATABASE" like MySQL.
7  -- In Oracle Live SQL, you already work inside your own schema.
8  -- So we directly create tables here (this covers "3.1 Database Creation").
9  --
10 --
11 -- 3.1 DATABASE AND TABLE CREATION
12 --
13 --
14 -- Drop tables if they already exist (safe re-run)
15 BEGIN
16 EXECUTE IMMEDIATE 'DROP TABLE Service CASCADE CONSTRAINTS';
17 EXCEPTION WHEN OTHERS THEN NULL;
18 END;
19 /
20 BEGIN
21 EXECUTE IMMEDIATE 'DROP TABLE Vehicle CASCADE CONSTRAINTS';
22 EXCEPTION WHEN OTHERS THEN NULL;
23 END;
24 /
25 BEGIN
26 EXECUTE IMMEDIATE 'DROP TABLE Customer CASCADE CONSTRAINTS';
27 EXCEPTION WHEN OTHERS THEN NULL;
28 END;
29 /
30 --
31 -- CUSTOMER TABLE
32 --
33 CREATE TABLE Customer (
34 customer_id NUMBER(5) PRIMARY KEY,
35 customer_name VARCHAR2(50) NOT NULL,
36 phone_number VARCHAR2(15) UNIQUE NOT NULL,
37 email VARCHAR2(50) UNIQUE,
38 address VARCHAR2(100)
39 );
40 --
41 -- VEHICLE TABLE
42 --
43 CREATE TABLE Vehicle (
44 vehicle_id NUMBER(5) PRIMARY KEY,
45 customer_id NUMBER(5) NOT NULL,
46 vehicle_number VARCHAR2(15) UNIQUE NOT NULL,
47 brand VARCHAR2(30) NOT NULL,
48 model VARCHAR2(30) NOT NULL,
49 make_year NUMBER(4) CHECK (make_year BETWEEN 1990 AND 2024),
50 CONSTRAINT fk_customer FOREIGN KEY (customer_id)
51 REFERENCES Customer (customer_id)
52 ON DELETE CASCADE
53 );

```

The left sidebar shows the 'My Schema' tree with the following tables listed:

- CUSTOMER
- SERVICE
- VEHICLE

The right sidebar shows 'SQL Tutorials' with the following content:

- Creating Tables: Databases for...**
An introduction to creating tables and the types of these available in Oracle Database.
Created 9 years ago
10 likes 17.6K executions
- Joining Tables: Databases for...**
An introduction to the join types available in Oracle Database.
Created 8 years ago

Screenshot 2 – Customer Table with Records

The screenshot shows the Oracle FreeSQL web interface. The main editor displays the following SQL code:

```

20 BEGIN
21 EXECUTE IMMEDIATE 'DROP TABLE Service CASCADE CONSTRAINTS';
22 EXCEPTION WHEN OTHERS THEN NULL;
23 END;
24 /
25 BEGIN
26 EXECUTE IMMEDIATE 'DROP TABLE Vehicle CASCADE CONSTRAINTS';
27 EXCEPTION WHEN OTHERS THEN NULL;
28 END;
29 /
30 --
31 -- CUSTOMER TABLE
32 --
33 CREATE TABLE Customer (
34 customer_id NUMBER(5) PRIMARY KEY,
35 customer_name VARCHAR2(50) NOT NULL,
36 phone_number VARCHAR2(15) UNIQUE NOT NULL,
37 email VARCHAR2(50) UNIQUE,
38 address VARCHAR2(100)
39 );
40 --
41 -- VEHICLE TABLE
42 --
43 CREATE TABLE Vehicle (
44 vehicle_id NUMBER(5) PRIMARY KEY,
45 customer_id NUMBER(5) NOT NULL,
46 vehicle_number VARCHAR2(15) UNIQUE NOT NULL,
47 brand VARCHAR2(30) NOT NULL,
48 model VARCHAR2(30) NOT NULL,
49 make_year NUMBER(4) CHECK (make_year BETWEEN 1990 AND 2024),
50 CONSTRAINT fk_customer FOREIGN KEY (customer_id)
51 REFERENCES Customer (customer_id)
52 ON DELETE CASCADE
53 );

```

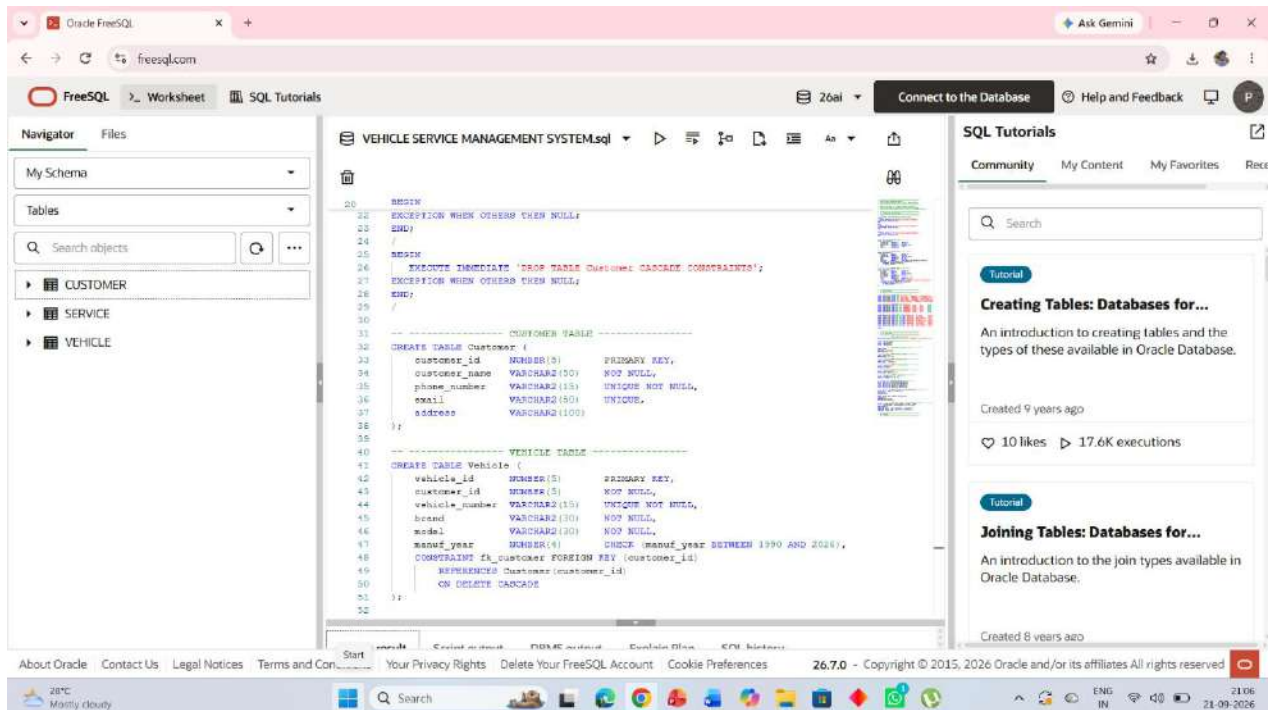
The left sidebar shows the 'My Schema' tree with the following tables listed:

- CUSTOMER
- SERVICE
- VEHICLE

The right sidebar shows 'SQL Tutorials' with the following content:

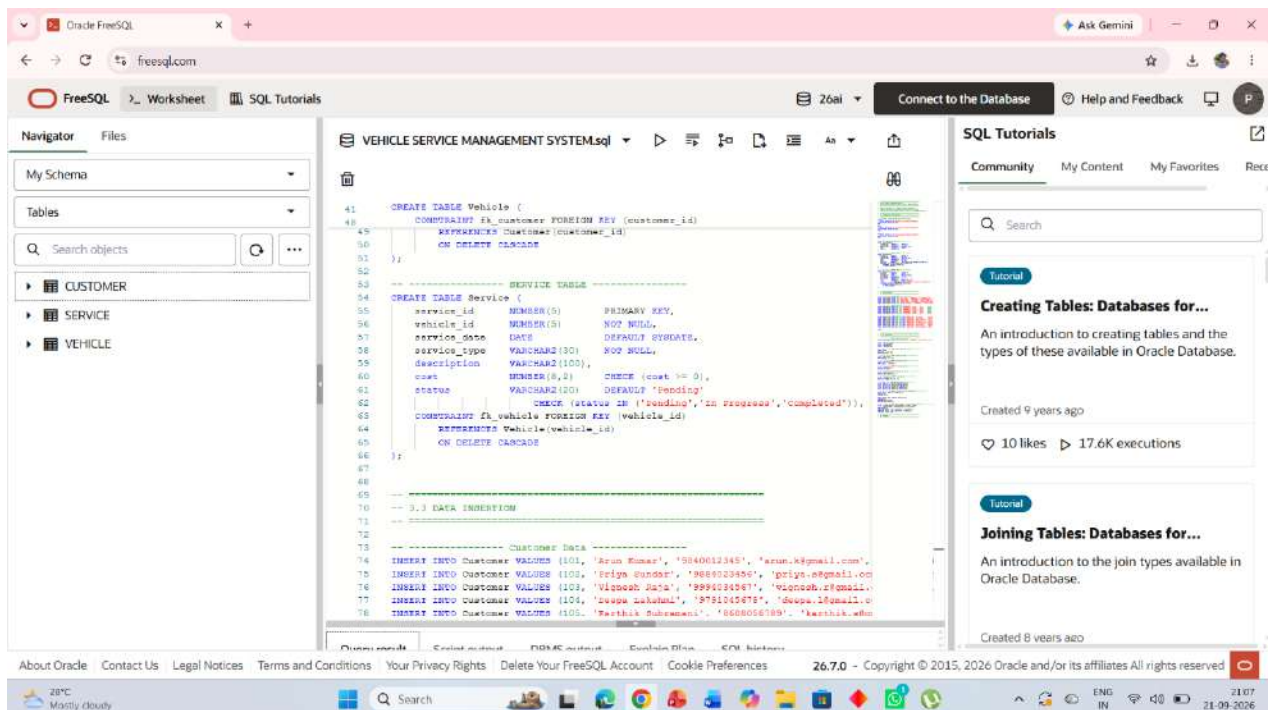
- Creating Tables: Databases for...**
An introduction to creating tables and the types of these available in Oracle Database.
Created 9 years ago
10 likes 17.6K executions
- Joining Tables: Databases for...**
An introduction to the join types available in Oracle Database.
Created 8 years ago

Screenshot 3 – Vehicle Table with Records



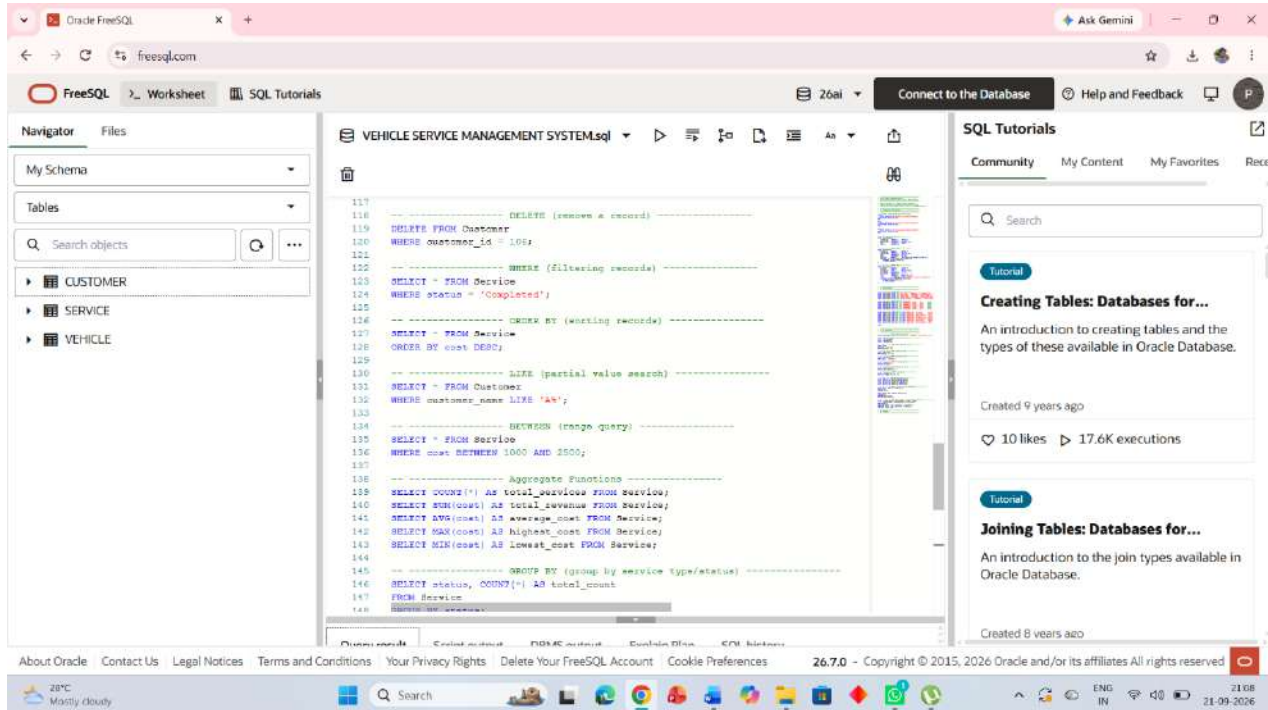
The screenshot shows the Oracle FreeSQL web interface. The left sidebar displays the 'My Schema' tree with tables 'CUSTOMER', 'SERVICE', and 'VEHICLE'. The main editor shows the SQL script for creating the 'VEHICLE' table. The script includes a comment 'VEHICLE SERVICE MANAGEMENT SYSTEM.sql', a 'BEGIN' statement, an 'EXCEPTION WHEN OTHERS THEN NULL;' block, an 'EXECUTE IMMEDIATE' statement to drop the table if it exists, another 'EXCEPTION WHEN OTHERS THEN NULL;' block, and an 'END;' statement. The 'CREATE TABLE Customer' statement is also visible, defining columns: customer_id (NUMBER(5) PRIMARY KEY), customer_name (VARCHAR2(100) NOT NULL), phone_number (VARCHAR2(15) UNIQUE NOT NULL), email (VARCHAR2(50) UNIQUE), and address (VARCHAR2(100)). The 'CREATE TABLE Vehicle' statement defines columns: vehicle_id (NUMBER(5) PRIMARY KEY), customer_id (NUMBER(5) NOT NULL), vehicle_number (VARCHAR2(15) UNIQUE NOT NULL), brand (VARCHAR2(30) NOT NULL), model (VARCHAR2(30) NOT NULL), and manufacture_year (NUMBER(4) CHECK (manufacture_year BETWEEN 1990 AND 2026)). It also includes a foreign key constraint 'CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES Customer (customer_id) ON DELETE CASCADE'.

Screenshot 4 – Service Table with Records



The screenshot shows the Oracle FreeSQL web interface. The left sidebar displays the 'My Schema' tree with tables 'CUSTOMER', 'SERVICE', and 'VEHICLE'. The main editor shows the SQL script for creating the 'SERVICE' table. The script includes a comment 'VEHICLE SERVICE MANAGEMENT SYSTEM.sql', a 'BEGIN' statement, an 'EXCEPTION WHEN OTHERS THEN NULL;' block, an 'EXECUTE IMMEDIATE' statement to drop the table if it exists, another 'EXCEPTION WHEN OTHERS THEN NULL;' block, and an 'END;' statement. The 'CREATE TABLE Service' statement defines columns: service_id (NUMBER(5) PRIMARY KEY), vehicle_id (NUMBER(5) NOT NULL), service_date (DATE DEFAULT SYSDATE), service_type (VARCHAR2(30) NOT NULL), description (VARCHAR2(100)), cost (NUMBER(8,2) CHECK (cost >= 0)), and status (VARCHAR2(20) DEFAULT 'pending'). It also includes a foreign key constraint 'CONSTRAINT fk_vehicle FOREIGN KEY (vehicle_id) REFERENCES Vehicle (vehicle_id) ON DELETE CASCADE'. Below the table creation, there is a section for data insertion: '3.3 DATA INSERTION'. The 'INSERT INTO Customer' statement is also visible, defining columns: customer_id, customer_name, phone_number, email, and address. The 'INSERT INTO Service' statement defines columns: service_id, vehicle_id, service_date, service_type, description, cost, and status. The script includes several 'INSERT INTO' statements for both 'Customer' and 'Service' tables.

Screenshot 5 – INSERT / UPDATE / DELETE Operations



The screenshot displays the Oracle FreeSQL web interface. The main editor shows a SQL script for the 'VEHICLE SERVICE MANAGEMENT SYSTEM.sql'. The script includes the following operations:

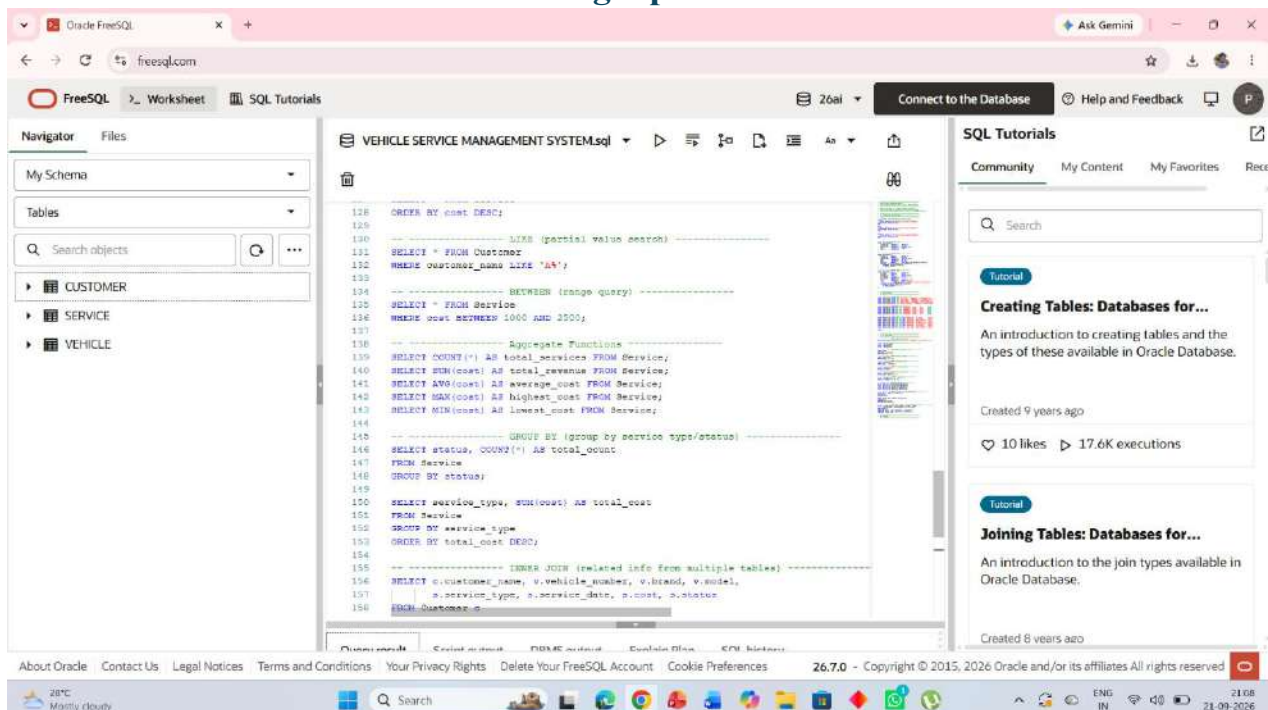
```

117 ----- DELETE (remove a record) -----
118 DELETE FROM Customer
119 WHERE customer_id = 100;
120
121 ----- WHERE (filtering records) -----
122 SELECT * FROM Service
123 WHERE status = 'Completed';
124
125 ----- ORDER BY (sorting records) -----
126 SELECT * FROM Service
127 ORDER BY cost DESC;
128
129 ----- LIKE (partial value search) -----
130 SELECT * FROM Customer
131 WHERE customer_name LIKE 'as';
132
133 ----- BETWEEN (range query) -----
134 SELECT * FROM Service
135 WHERE cost BETWEEN 1000 AND 2000;
136
137 ----- Aggregate Functions -----
138 SELECT COUNT(*) AS total_services FROM Service;
139 SELECT SUM(cost) AS total_revenue FROM Service;
140 SELECT AVG(cost) AS average_cost FROM Service;
141 SELECT MAX(cost) AS highest_cost FROM Service;
142 SELECT MIN(cost) AS lowest_cost FROM Service;
143
144 ----- GROUP BY (group by service type/status) -----
145 SELECT status, COUNT(*) AS total_count
146 FROM Service
147 GROUP BY status;
148
149 ----- INNER JOIN (related info from multiple tables) -----
150 SELECT c.customer_name, v.vehicle_number, v.brand, v.model,
151        s.service_type, s.service_date, s.cost, s.status
152 FROM Customer c
153 JOIN Service s ON c.customer_id = s.customer_id;

```

The interface also shows a Navigator panel on the left with a tree view of the database schema (My Schema, Tables, CUSTOMER, SERVICE, VEHICLE). The right panel displays SQL Tutorials, including 'Creating Tables: Databases for...' and 'Joining Tables: Databases for...'.

Screenshot 6 – SELECT and Filtering Operations



The screenshot displays the Oracle FreeSQL web interface. The main editor shows a SQL script for the 'VEHICLE SERVICE MANAGEMENT SYSTEM.sql'. The script includes the following operations:

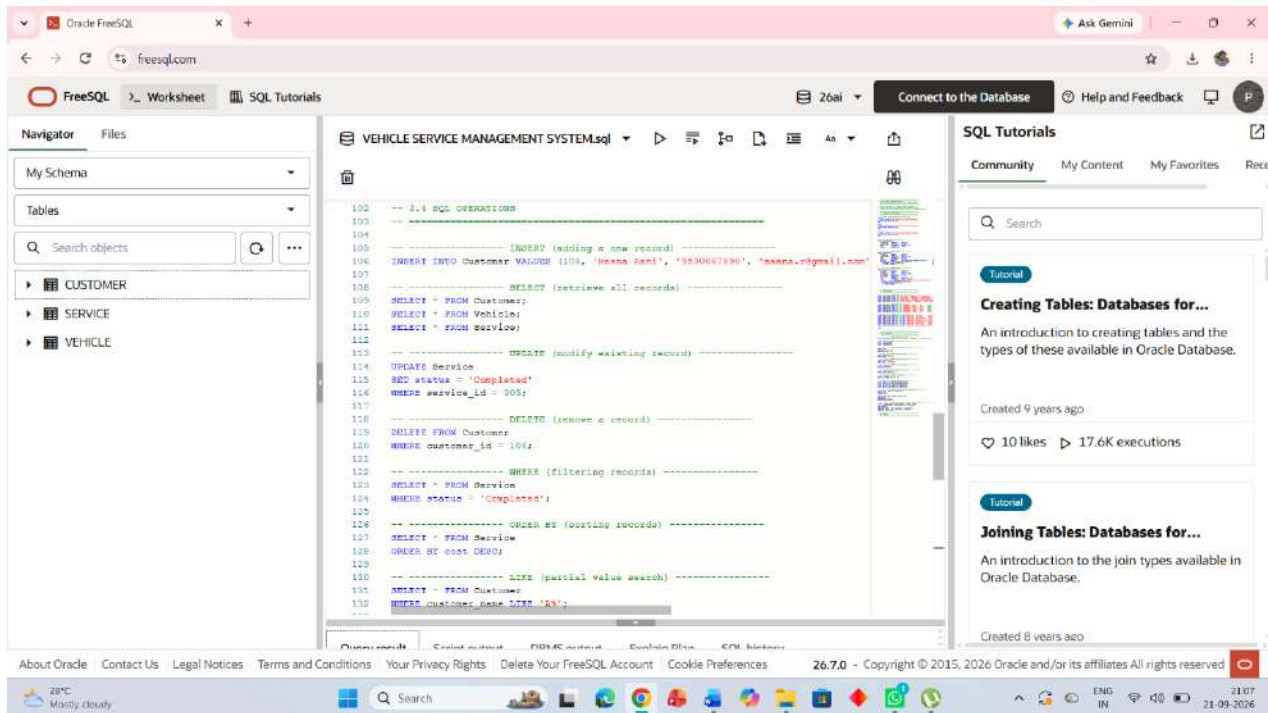
```

128 ORDER BY cost DESC;
129
130 ----- LIKE (partial value search) -----
131 SELECT * FROM Customer
132 WHERE customer_name LIKE 'as';
133
134 ----- BETWEEN (range query) -----
135 SELECT * FROM Service
136 WHERE cost BETWEEN 1000 AND 2000;
137
138 ----- Aggregate Functions -----
139 SELECT COUNT(*) AS total_services FROM Service;
140 SELECT SUM(cost) AS total_revenue FROM Service;
141 SELECT AVG(cost) AS average_cost FROM Service;
142 SELECT MAX(cost) AS highest_cost FROM Service;
143 SELECT MIN(cost) AS lowest_cost FROM Service;
144
145 ----- GROUP BY (group by service type/status) -----
146 SELECT status, COUNT(*) AS total_count
147 FROM Service
148 GROUP BY status;
149
150 SELECT service_type, SUM(cost) AS total_cost
151 FROM Service
152 GROUP BY service_type
153 ORDER BY total_cost DESC;
154
155 ----- INNER JOIN (related info from multiple tables) -----
156 SELECT c.customer_name, v.vehicle_number, v.brand, v.model,
157        s.service_type, s.service_date, s.cost, s.status
158 FROM Customer c
159 JOIN Service s ON c.customer_id = s.customer_id;

```

The interface also shows a Navigator panel on the left with a tree view of the database schema (My Schema, Tables, CUSTOMER, SERVICE, VEHICLE). The right panel displays SQL Tutorials, including 'Creating Tables: Databases for...' and 'Joining Tables: Databases for...'.

Screenshot 7 – Aggregate Functions



The screenshot shows the Oracle FreeSQL web interface. The main editor displays a script titled "VEHICLE SERVICE MANAGEMENT SYSTEM.sql" with the following SQL queries:

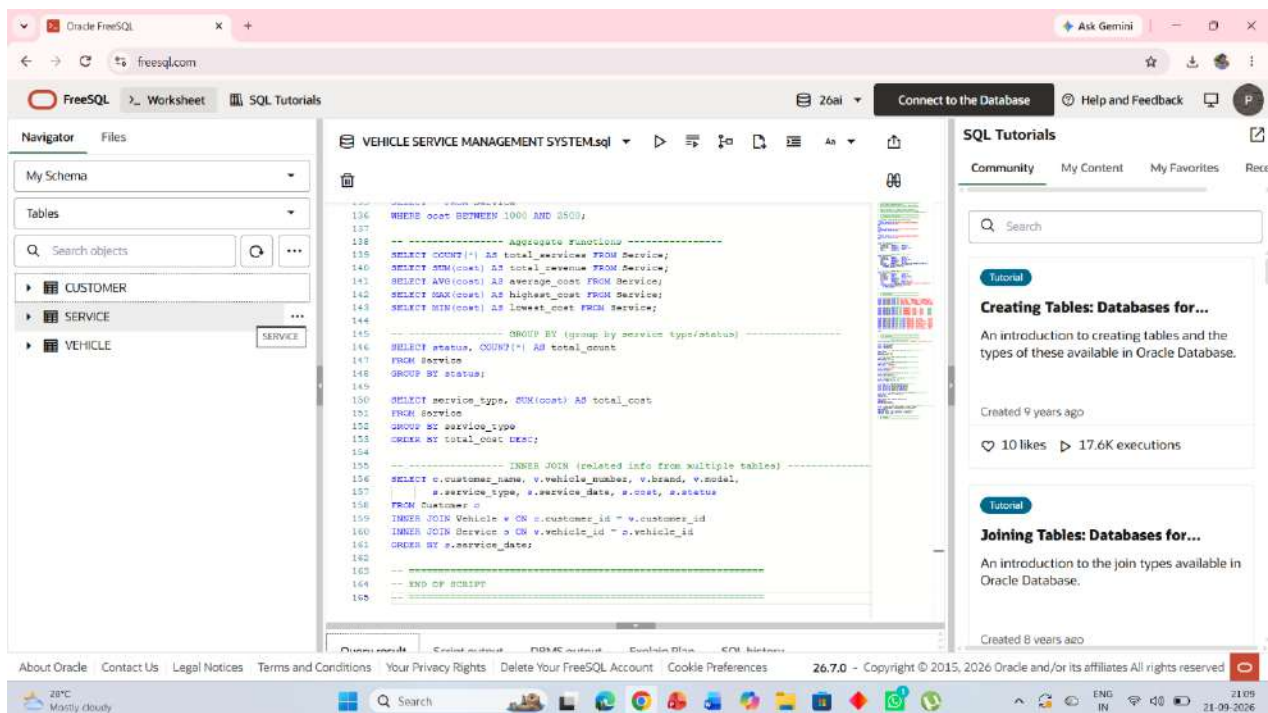
```

-- 3.4 SQL OPERATIONS
--
-- INSERT (adding a new record)
INSERT INTO Customer VALUES (109, 'Sneha Rani', '9500067890', 'sneha.r@gmail.com');
--
-- SELECT (retrieve all records)
SELECT * FROM Customer;
SELECT * FROM Vehicle;
SELECT * FROM Service;
--
-- UPDATE (modify existing record)
UPDATE Service
SET status = 'Completed'
WHERE service_id = 505;
--
-- DELETE (remove a record)
DELETE FROM Customer
WHERE customer_id = 104;
--
-- WHERE (filtering records)
SELECT * FROM Service
WHERE status = 'Completed';
--
-- ORDER BY (sorting records)
SELECT * FROM Service
ORDER BY cost DESC;
--
-- LIKE (partial value search)
SELECT * FROM Customer
WHERE customer_name LIKE 'AN';

```

The interface includes a Navigator panel on the left showing a schema with tables CUSTOMER, SERVICE, and VEHICLE. The right sidebar displays "SQL Tutorials" with links to "Creating Tables: Databases for..." and "Joining Tables: Databases for...". The bottom status bar shows the version as 26.7.0 and copyright information for 2015, 2026 Oracle.

Screenshot 8 – GROUP BY Operation



The screenshot shows the Oracle FreeSQL web interface with the same "VEHICLE SERVICE MANAGEMENT SYSTEM.sql" script. The SQL queries are as follows:

```

-- WHERE clause example
WHERE cost BETWEEN 1000 AND 2500;
--
-- Aggregate Functions
SELECT COUNT(*) AS total_services FROM Service;
SELECT MIN(cost) AS total_revenue FROM Service;
SELECT AVG(cost) AS average_cost FROM Service;
SELECT MAX(cost) AS highest_cost FROM Service;
SELECT MIN(cost) AS lowest_cost FROM Service;
--
-- GROUP BY (group by service type/status)
SELECT status, COUNT(*) AS total_count
FROM Service
GROUP BY status;
--
-- ROLLUP (total count)
SELECT service_type, ROLLUP(COUNT(*) AS total_count
FROM Service
GROUP BY service_type
ORDER BY total_count DESC;
--
-- INNER JOIN (related info from multiple tables)
SELECT c.customer_name, v.vehicle_number, v.brand, v.model,
s.service_type, s.service_date, s.cost, s.status
FROM Customer c
INNER JOIN Vehicle v ON c.customer_id = v.customer_id
INNER JOIN Service s ON v.vehicle_id = s.vehicle_id
ORDER BY s.service_date;
--
-- END OF SCRIPT

```

The interface elements are consistent with Screenshot 7, including the Navigator panel, SQL Tutorials sidebar, and bottom status bar.

Screenshot 9 – INNER JOIN Operation

The screenshot displays the Oracle FreeSQL web application interface. The main editor shows an SQL script titled "VEHICLE SERVICE MANAGEMENT SYSTEM.sql". The script includes the following SQL statements:

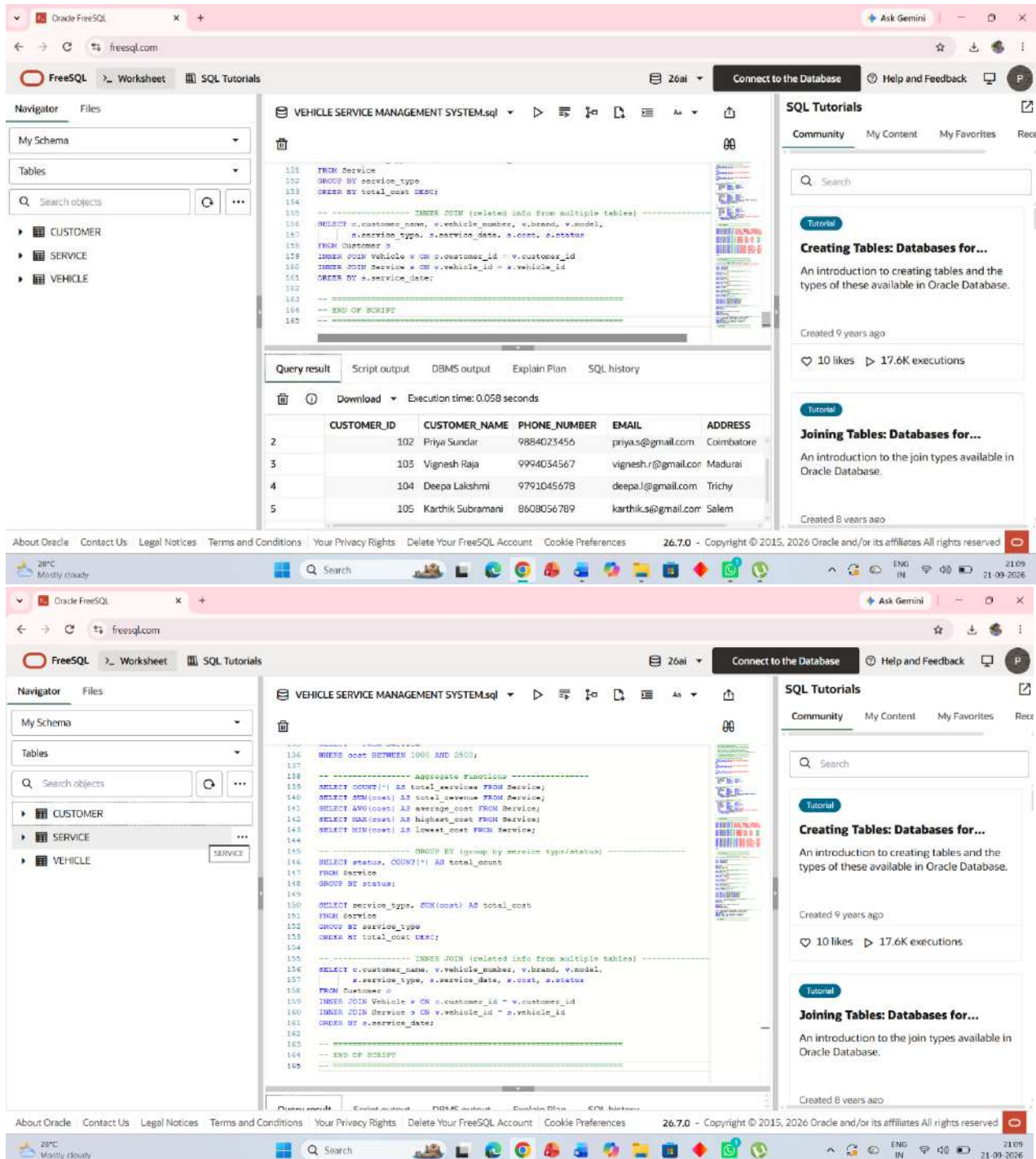
```

136 WHERE cost BETWEEN 1000 AND 2500;
137
138 -- Aggregate Functions -----
139 SELECT COUNT(*) AS total_service FROM Service;
140 SELECT SUM(cost) AS total_revenue FROM Service;
141 SELECT AVG(cost) AS average_cost FROM Service;
142 SELECT MAX(cost) AS highest_cost FROM Service;
143 SELECT MIN(cost) AS lowest_cost FROM Service;
144
145 -- GROUP BY (group by service type/status) -----
146 SELECT status, COUNT(*) AS total_count
147 FROM Service
148 GROUP BY status;
149
150 SELECT service_type, SUM(cost) AS total_cost
151 FROM Service
152 GROUP BY service_type
153 ORDER BY total_cost DESC;
154
155 -- INNER JOIN (related info from multiple tables) -----
156 SELECT c.customer_name, v.vehicle_make, v.brand, v.model,
157        s.service_type, s.service_date, s.cost, s.status
158 FROM Customer c
159 INNER JOIN Vehicle v ON c.customer_id = v.customer_id
160 INNER JOIN Service s ON v.vehicle_id = s.vehicle_id
161 ORDER BY s.service_date;
162
163 --
164 -- END OF SCRIPT
165

```

The interface also features a Navigator panel on the left showing a schema tree with tables: CUSTOMER, SERVICE, and VEHICLE. The SERVICE table is currently selected. On the right, there is a "SQL Tutorials" sidebar with two tutorial cards: "Creating Tables: Databases for..." and "Joining Tables: Databases for...". The bottom of the page shows a footer with "About Oracle", "Contact Us", "Legal Notices", "Terms and Conditions", "Your Privacy Rights", "Delete Your FreeSQL Account", and "Cookie Preferences". The copyright notice states "2015, 2026 Oracle and/or its affiliates All rights reserved".

Screenshot 10 – Final Database / Query Results



The screenshot displays the FreeSQL web interface for a database named 'VEHICLE SERVICE MANAGEMENT SYSTEM.sql'. The interface includes a Navigator on the left, a central query editor, and a right sidebar with SQL Tutorials.

Navigator: Shows the database structure with tables: CUSTOMER, SERVICE, and VEHICLE.

Query Editor: Contains the following SQL script:

```

151 FROM Service
152 GROUP BY service_type
153 ORDER BY total_cost DESC;
154
155 -- ----- INNER JOIN (related info from multiple tables) -----
156 SELECT c.customer_name, v.vehicle_number, v.brand, v.model,
157        s.service_type, s.service_date, s.cost, s.status
158 FROM Customer c
159 INNER JOIN Vehicle v ON c.customer_id = v.customer_id
160 INNER JOIN Service s ON v.vehicle_id = s.vehicle_id
161 ORDER BY s.service_date;
162
163 -- -----
164 -- END OF SCRIPT
165

```

Query result: Shows the execution time as 0.058 seconds and a table of results:

	CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS
2	102	Priya Sundar	9884023456	priya.s@gmail.com	Coimbatore
3	103	Vignesh Raja	9994034567	vignesh.r@gmail.com	Madurai
4	104	Deepa Lakshmi	9791045678	deepa.l@gmail.com	Trichy
5	105	Karthik Subramani	8608056789	karthik.s@gmail.com	Salem

SQL Tutorials: The sidebar shows two tutorials: 'Creating Tables: Databases for...' and 'Joining Tables: Databases for...'. Both tutorials are created 9 years ago and have 10 likes and 17.6K executions.

5 Technical Explanation – 5 Marks

Database Design

The database is split into three separate tables to avoid data redundancy and follow normalization principles. The Customer table stores customer-specific details that do not change often. The Vehicle table stores vehicle details separately because one customer can own multiple vehicles — storing vehicle data in the Customer table would duplicate customer information for every vehicle. Similarly, the Service table is separated because one vehicle can undergo many service visits over time; keeping service records in their own table allows unlimited service history without altering the Vehicle table's structure.

Keys and Constraints

Each table has a PRIMARY KEY (customer_id, vehicle_id, service_id) to uniquely identify every row. FOREIGN KEYS link Vehicle to Customer and Service to Vehicle, enforcing that a vehicle cannot exist without a valid customer, and a service record cannot exist without a valid vehicle. NOT NULL constraints ensure essential fields are always filled, UNIQUE constraints prevent duplicate phone numbers, emails, and vehicle numbers, and CHECK constraints restrict values such as manufacturing year, cost, and status to valid ranges/options. ON DELETE CASCADE ensures that removing a customer or vehicle automatically removes their dependent records, keeping the database consistent.

SQL Operations

INSERT statements were used to add new customer, vehicle, and service records. SELECT statements retrieve data from one or more tables. UPDATE statements were used to change existing values, such as marking a service as 'Completed'. DELETE statements remove records that are no longer needed, such as a mistakenly added customer, with the foreign key relationships and ON DELETE CASCADE ensuring related records are handled correctly.

Data Retrieval

WHERE was used to filter rows based on a condition (e.g., status = 'Completed'). ORDER BY sorts the result set, such as arranging services by cost in descending order. LIKE performs partial/pattern matching, useful for searching customers by name. BETWEEN retrieves rows within a numeric or date range, such as services within a certain cost bracket. GROUP BY aggregates rows that share a common value (like status or service_type) so that summary statistics can be calculated for each group.

Aggregate Functions and Joins

Aggregate functions — COUNT(), SUM(), AVG(), MAX(), and MIN() — summarize numeric data across many rows, which is useful for reporting total revenue, average service cost, and the number of services performed. INNER JOIN was used to combine the Customer, Vehicle, and Service tables using their key relationships, allowing a single query to display which customer owns which vehicle and what services were performed on it, rather than looking up each table separately.

6 Conclusion – 5 Marks

Conclusion

The Vehicle Service Management Database successfully models a real-world vehicle service centre using three well-structured, related tables: Customer, Vehicle, and Service. Primary keys, foreign keys, and constraints were used to maintain data integrity and enforce meaningful relationships between customers, their vehicles, and service history. Core SQL operations — INSERT, SELECT, UPDATE, and DELETE — were implemented and demonstrated, along with data retrieval techniques such as WHERE, ORDER BY, LIKE, BETWEEN, and GROUP BY. Aggregate functions and INNER JOIN queries were used to generate meaningful business insights such as total revenue, average service cost, and combined customer-vehicle-service reports. Through this project, key database administration concepts — schema design, normalization, constraints, and SQL querying — were learned and applied practically using SQL.

FUTURE ENHANCEMENT

- Customer login and authentication.
- Online service appointment booking.
- Automated service reminders.
- Service history reports.
- Invoice generation.
- Mechanic assignment and management.
- Integration with a web or mobile application.
- Automated database backup and recovery.